

TorusNet 4.0 Implementation

L'Architech (Gabriel Breton-Breault)

★ Introduction — Main Language of the Protocol: C

TorusNet is implemented in the **C programming language**, which serves as the technical foundation of the entire protocol. This choice is deliberate and directly aligned with the structural, directional, and hardware-level requirements of the project. TorusNet processes massive binary volumes, traverses internal geometric structures, and relies on precise memory control — characteristics that make C uniquely suited for this type of directional architecture.

The protocol must handle amplified binary seeds, large directional vectors, internal private-torus structures, directional portals, and coherence buffers. C provides direct access to pointers, memory allocation, data structures, binary blocks, and sequential memory operations, all of which are essential for a protocol built on directional logic.

Beyond memory control, C delivers the **raw performance** TorusNet requires. The protocol must amplify a 30-bit seed into a sequence of several hundred megabytes, traverse a private torus, validate topological coherence, and generate unique directional keys for every request. C ensures fast execution, low latency, efficient buffer management, and compatibility with future hardware implementations.

This choice also supports TorusNet's long-term vision: the protocol is designed to be **photon-ready**. Photonic architectures and directional processors typically rely on low-level languages such as C or C++. Using C facilitates future transitions toward advanced modules such as the Glass Fortress and SMD/VSMD photonic systems.

Finally, TorusNet must remain independent from any software abstraction layer. It must not rely on a framework, virtual machine, runtime environment, or garbage collector. C, being minimalistic and autonomous, is perfectly suited for an experimental protocol whose security is based on structure and geometry rather than traditional cryptography.

Chapter 1 — Random Generation of the Private Torus

The private torus is a unique directional structure composed of **1,000,000 cells** generated entirely at random. It is stored on the hard drive in the form of encrypted files and represents the core security element of TorusNet: it is non-transmissible, non-derivable, and impossible to reconstruct without direct access to its internal files.

The private torus is built from three main components:

1. the **normal-bit file**,
2. the **directional-movement and exponential-jump file**,
3. the **inaccessible cells** (10% of the torus), which may also serve as starting cells.

All files are loaded into RAM during use and are **read in parallel** while the torus is being traversed.

1.1 — Normal-Bit File (Pure Binary)

The normal-bit file contains a pure binary sequence representing the **1,000,000 cells** of the torus. It is generated randomly and stored as an encrypted binary file on the hard drive.

Characteristics:

- pure binary file,
- encrypted at rest,
- loaded into RAM during use,
- used for directional reading,
- **10% of the cells are marked as inaccessible via exponential jumps.**

These inaccessible cells play a special role: they may serve as **starting cells**, because exponential jumps can never reach them. They form directional origin points that cannot be accessed through linear traversal.

1.2 — Directional-Movement File

The second file contains the **four possible movements per cell**, along with their associated **exponential jumps**. Each entry follows a compact structure:

Code

101 2³⁴, 010 3⁵, 110 2⁴, 111 3⁵, ...

Interpretation:

- The **three bits** (e.g., 101, 010, 110, 111) indicate the **direction**:
 - up
 - down
 - left
 - right
 - diagonals
- The exponent (e.g., 2^{34} , 3^5) indicates the **exponential jump** to perform inside the torus.

This file is also:

- encrypted on disk,
- loaded into RAM during use,
- read in parallel with the normal-bit file.

1.3 — Toroidal Behavior of the Canvas

The private torus behaves like a toroidal surface. Directional movements follow these rules:

Directional Movements:

- moving past the top → reappears at the bottom
- moving past the bottom → reappears at the top
- moving past the left → reappears at the right
- moving past the right → reappears at the left

This ensures perfect continuity with no borders.

Exponential Jumps:

Exponential jumps are performed **linearly** within the normal-bit file:

- advance by N^E positions,
- if the end of the file is reached → wrap to the beginning,
- the jump is independent of direction.

1.4 — Cell IDs (Hexadecimal)

Some cells in the torus — specifically the **starting cells** — contain a **directional ID** encoded in **hexadecimal**. This ID encodes:

- the direction,
- the nature of the cell,
- its role in directional coherence.

Important:

- **Not all cells have an ID.**
- Only **starting cells** (including the 10% inaccessible cells) contain an identifier.
- All other cells are internal cells without IDs.

This distinction reinforces the directional structure of the torus and prevents any external mapping or derivation.

1.5 — Result: A Unique, Non-Derivable, and Unbreakable Private Torus

Through this generation process:

- the torus contains **1,000,000 cells**,
- 90% are reachable via exponential jumps,
- 10% are unreachable and serve as origin points,
- hexadecimal IDs define starting cells,
- movements are toroidal,
- jumps are linear,
- both files are read in parallel,
- the torus cannot be reconstructed without its internal files.

The private torus becomes a fully autonomous directional structure, non-transmissible and essential to TorusNet's security model.

Chapter 2 — Seed Generation, File Reconstruction, and the Removal of the Public Torus

In TorusNet, the **30-bit seed** is not generated before traversal. Instead, it is produced **during** the traversal of the private torus, at the same time as the file to be encoded is reconstructed. This dynamic mechanism makes the public torus completely obsolete in modern versions of the protocol.

This chapter explains how the program:

1. generates the seed,
2. reconstructs the file during traversal,
3. uses the amplified bits to move inside the private torus,
4. climbs back to the original seed,
5. encodes the arrival cell,
6. produces the final directional key.

2.1 — Seed Generation During Traversal

Unlike earlier versions of the protocol, the seed is not generated beforehand. It is produced **while** the program traverses the private torus and reconstructs the file.

The final seed is therefore directly tied to:

- the path taken,
- the normal bits read,
- the ghost bits used,
- the chosen directions,
- the arrival cell.

This makes every seed **unique, non-reproducible**, and **impossible to derive** without the private torus.

2.2 — Exponential Amplification: $30\text{bits}^{2^{28}} \approx 1 \text{ GB}$

Once generated, the seed is amplified according to:

$$30 \text{ bits}^{2^{28}}$$

This exponential amplification produces **approximately 1 GB of directional bits**.

These amplified bits are used to:

- guide movement inside the private torus,
- feed exponential jumps,
- reconstruct the file,
- generate movement vectors,
- maintain topological coherence.

The seed becomes a **massive directional source**, fully capable of replacing the public torus.

2.3 — File Reconstruction During Traversal

When a file needs to be encoded, the program:

1. selects a **starting cell**,
2. begins traversing the private torus,
3. reads at each step:
 - either a **normal bit**,
 - or a **ghost bit**,
4. encodes the chosen direction in binary (3 bits),
5. appends the bit to the reconstructed file.

The reconstruction happens **in real time**, during traversal.

If the reconstructed file is **smaller** than the amount of data produced by the amplified seed, the program **fills the remaining space** with random 0s and 1s.

Thus, the reconstructed file always reaches the maximum size the seed can provide.

2.4 — Movement Inside the Private Torus Using Amplified Bits

The amplified bits (≈ 1 GB) directly drive movement inside the private torus:

- they determine directions,
- they feed exponential jumps,
- they guide the reading of normal or ghost bits,
- they define the entire traversal path.

The private torus becomes a directional space fully controlled by the amplified seed.

2.5 — Climbing Back to the 30-Bit Seed

Once reconstruction is complete, the program **climbs back** through the generated sequence:

- from amplified bits,
- to original bits,
- back to the 30-bit seed.

This reverse traversal allows the program to recover the **directional origin point**, required to generate the final key.

2.6 — Encoding the Arrival Cell

The arrival cell in the private torus is also encoded into the key:

- its hexadecimal ID,
- its position,
- its final direction.

This allows the server to verify:

- the coherence of the traversal,
- the validity of the seed,
- the integrity of the key.

2.7 — Removal of the Public Torus

With this mechanism:

- the seed generates its own directional space,
- the amplified bits fully replace the public torus,
- the private torus provides all structural information,
- internal files define all movement rules,
- exponential jumps provide traversal dynamics.

The public torus becomes **useless, redundant**, and is **removed** in TorusNet 4.0+.

2.8 — Final Directional Key

The final key contains:

- the 30-bit seed,
- the directions taken,
- the normal and ghost bits used,
- the arrival cell,
- the topological coherence.

It is **unique, non-reproducible**, and **impossible to derive** without:

- the private torus,
- the internal files,
- the amplified seed,
- the exact traversal path.

Chapter 3 — Directional Movements, Exponential Jumps, and Navigation Inside the Private Torus

The private torus is a closed directional space composed of **1,000,000 cells**, each defined by normal bits, ghost bits, directional rules, and exponential jumps. This chapter explains how the program moves inside the torus, how directional rules are applied, and why this structure makes TorusNet impossible to derive or map externally.

3.1 — Two Reading Sources: Normal Bits and Ghost Bits

During traversal of the private torus, the program reads two types of data:

Normal Bits

- come from the pure binary file,
- represent the raw material of the torus,
- are used to reconstruct the file being encoded,
- exist in **every cell**.

Ghost Bits

- come from the movement file,
- represent directions and exponential jumps,
- are not part of the reconstructed file,

- serve only to guide navigation.

At each step, the program decides:

- to read a normal bit,
- or to read a ghost bit,
- or to combine both.

This parallel reading produces a unique directional trajectory.

3.2 — Directions: 3 Bits per Movement

Each cell in the torus contains **four possible movements**, encoded using **3 bits**:

Code

000 = up

001 = down

010 = left

011 = right

100 = diagonal up-left

101 = diagonal up-right

110 = diagonal down-left

111 = diagonal down-right

These directions are read from the movement file.

The program encodes each chosen direction in binary inside the reconstructed file, allowing the server to verify traversal coherence.

3.3 — Toroidal Behavior of the Private Torus

The private torus behaves like a toroidal surface:

- moving past the top → reappears at the bottom,
- moving past the bottom → reappears at the top,
- moving past the left → reappears at the right,
- moving past the right → reappears at the left.

This ensures:

- no borders,
- perfect continuity,
- impossibility of determining the torus size from outside.

3.4 — Exponential Jumps

Each direction is paired with an **exponential jump**, for example:

Code

101 2³⁴

010 3⁵

110 2⁴

The exponent indicates how many cells the program must skip **linearly** inside the normal-bit file.

Rules of Exponential Jumps:

- the jump is independent of direction,
- it advances through the binary file,
- reaching the end → wraps to the beginning,
- it can never reach the **10% inaccessible cells**.

These inaccessible cells serve as **origin points** and **starting cells**.

3.5 — Inaccessible Cells and Starting Cells

The torus contains:

- **90% reachable cells**,
- **10% unreachable cells** (via exponential jumps).

Unreachable cells:

- can only be reached through directional movement,
- can never be reached through exponential jumps,
- serve as **starting cells**,

- contain a **hexadecimal ID**,
- are used to generate unique directional keys.

Other cells do not have IDs.

3.6 — Directional Navigation and File Reconstruction

During traversal:

1. the program reads a normal or ghost bit,
2. chooses a direction,
3. encodes this direction in binary,
4. applies the exponential jump,
5. appends the bit to the reconstructed file.

If the reconstructed file is smaller than the amplified seed capacity, the program fills the remaining space with random 0s and 1s.

3.7 — Arrival, Reverse Traversal, and Coherence

At the end of traversal:

- the **arrival cell** is encoded into the key,
- the program **climbs back** through the amplified bits to the original 30-bit seed,
- the seed becomes the basis of the directional key,
- the server verifies traversal coherence.

3.8 — Result: Navigation Impossible to Reproduce

Thanks to this structure:

- every traversal is unique,
- every key is non-reproducible,
- the private torus cannot be mapped,
- exponential jumps prevent any analysis,
- inaccessible cells guarantee inviolable origin points.

The private torus becomes a fully autonomous directional space, impossible to derive without access to its internal files.